UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 18

Taller 2 — Resolución de problemas

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 Objetivos de la sesión **2**

2 Dinámica del taller **2**

3 Problema 1: El profesor y los nombres **3**
3.1 Enunciado completo 3
3.2 Parte 1 — Guía de pizarrón 3
3.3 Análisis de la solución 3
3.4 Solución en C++ 4

4 Problema 2: Once Cifras **6**
4.1 Enunciado 6
4.2 Parte 1 — Guía de pizarrón 6
4.3 Análisis de la solución 6
4.4 Solución en C++ 7

5 Materiales y recursos **8**

Objetivos de la sesión

Al finalizar la sesión el alumno será capaz de:

1. Buscar el elemento más cercano en una lista hacia la izquierda y hacia la derecha con un recorrido lineal.
2. Construir una cadena de salida que depende de una distancia calculada en tiempo de ejecución.
3. Reconocer cuándo la fuerza bruta es la estrategia correcta (problema de tamaño fijo y pequeño).
4. Usar `substr` y `stoll` para procesar dígitos de un número leído como cadena.

90 minutos. Distribución sugerida:
5 min — presentación del taller
10 min — **Problema 1, Parte 1:** revisión guiada en pizarrón
30 min — **Problema 1, Parte 2:** trabajo en equipo + envío
8 min — **Problema 2, Parte 1:** revisión guiada en pizarrón
27 min — **Problema 2, Parte 2:** trabajo en equipo + envío
10 min — cierre: discusión de soluciones

Los alumnos trabajan en equipos de dos o tres personas y usan el cuaderno Jupyter (`sesiones/sesion18_taller_2.ipynb`) para probar localmente.

Dinámica del taller

Para cada problema, sigue este orden en el pizarrón:

1. Lee el enunciado en voz alta; detente en cada párrafo y pregunta “¿qué nos dan?” y “¿qué nos piden?”.
2. Escribe la entrada del ejemplo tal como aparece; etiqueta cada dato (“esta línea es n”, “estos son los nombres”, etc.).
3. Traza el procesamiento paso a paso con los números concretos.
4. Escribe la salida obtenida y verifica que coincide con el enunciado.
5. Lanza la pregunta de cierre sin responderla: deja esa generalización para el equipo.

Verifica que todos los alumnos tengan cuenta en el juez usado antes de empezar la Parte 2.

Problema 1: El profesor y los nombres

• Enunciado completo

Dados n alumnos en fila, algunos con nombre conocido y otros marcados con `?`, responder m consultas: ¿cómo llamarías al alumno en la posición p?

- Si la posición tiene nombre conocido: imprimir el nombre.
- Si el conocido más cercano está a distancia d a la izquierda: "`derecha` $\times d$ `de Nombre`".
- Si está a la derecha: "`izquierda` $\times d$ `de Nombre`".
- Si hay dos a igual distancia (uno a cada lado): "`justo enmedio de NombreIzq y NombreDer`".

• Parte 1 — Guía de pizarrón

1. Dibuja la fila con posiciones numeradas del 1 al 10:

A	?	?	B	?	?	?	C	?	?
1	2	3	4	5	6	7	8	9	10

2. **Consulta** $p = 3$**:** "¿Quién es el más cercano?" Señala a la izquierda: A en pos. 1, distancia 2. Señala a la derecha: B en pos. 4, distancia 1. B es más cercano. B está a la *derecha* de la posición 3, por eso lo llamamos desde su *izquierda*. Salida: "`izquierda de B`".
3. **Consulta** $p = 8$**:** nombre conocido (C). Salida: "`C`".
4. **Consulta** $p = 6$**:** B en pos. 4, distancia 2. C en pos. 8, distancia 2. Igual $\rightarrow$ enmedio. Salida: "`justo enmedio de B y C`" (primero el de la izquierda).
5. **Consulta** $p = 10$**:** solo C a la izquierda, distancia 2. No hay nadie a la derecha. C está a la *izquierda* de la posición 10, entonces nos movemos hacia su *derecha*. Salida: "`derecha derecha de C`".
6. Pregunta de cierre: "¿Cómo generalizamos esto para cualquier fila y cualquier consulta?" — dejar para el equipo.

• Análisis de la solución

El algoritmo para cada consulta p:

1. Si `nombres[p-1] != "?"`: imprimir el nombre y continuar.
2. Recorrer hacia la izquierda desde $p - 2$ hasta 0; el primer nombre conocido encontrado es el más cercano a la izquierda (distancia $d_L = p - 1 - i$, donde i es su índice en el arreglo).

3. Recorrer hacia la derecha desde p hasta $n-1$; el primer nombre conocido encontrado es el más cercano a la derecha (distancia $d_R = i - (p-1)$).
4. Si solo existe d_L: imprimir "`derecha`" d_L veces + "`de Nombre`".
5. Si solo existe d_R: imprimir "`izquierda`" d_R veces + "`de Nombre`".
6. Si $d_L < d_R$: imprimir "`derecha`" d_L veces + "`de NombreIzq`".
7. Si $d_R < d_L$: imprimir "`izquierda`" d_R veces + "`de NombreDer`".
8. Si $d_L = d_R$: imprimir "`justo enmedio de NombreIzq y NombreDer`".

• Solución en C++

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<string> nom(n);
    for (auto& s : nom) cin >> s;

    int m;
    cin >> m;
    while (m--) {
        int p;
        cin >> p;
        p--;  // pasar a índice 0

        if (nom[p] != "?") {
            cout << nom[p] << "\n";
            continue;
        }

        int dL = -1, dR = -1;
        string nL, nR;

        for (int i = p - 1; i >= 0; i--) {
            if (nom[i] != "?") { dL = p - i; nL = nom[i]; break; }
        }
        for (int i = p + 1; i < n; i++) {
            if (nom[i] != "?") { dR = i - p; nR = nom[i]; break; }
        }

        if (dL == -1 || (dR != -1 && dR < dL)) {
            for (int i = 0; i < dR; i++) cout << "izquierda ";
```

```
            cout << "de " << nR << "\n";
        } else if (dR == -1 || dL < dR) {
            for (int i = 0; i < dL; i++) cout << "derecha ";
            cout << "de " << nL << "\n";
        } else {
            cout << "justo enmedio de " << nL << " y " << nR << "\n";
        }
    }
}
```

¿Por qué "izquierda de B" y no "derecha de A" en la posición 3?
B está a distancia 1 y A está a distancia 2. B es el más cercano. B está a la derecha de la posición 3, así que el alumno está a la izquierda de B.

¿Puede haber más de dos nombres a la misma distancia mínima?
No. Como hay un solo alumno por posición, solo puede haber un nombre conocido a distancia d a la izquierda y otro a distancia d a la derecha. El caso de dos equidistantes siempre es uno a cada lado.

¿Qué pasa si todos los alumnos menos uno son `?`?
El problema garantiza al menos un nombre conocido, así que siempre habrá al menos un d_L o d_R definido. No hay caso sin respuesta.

Problema 2: Once Cifras

• Enunciado

Dado un número de 11 cifras, partirlo en tres subcadenas contiguas y sumar las tres partes interpretadas como enteros. Encontrar la partición que minimiza esa suma.

• Parte 1 — Guía de pizarrón

1. Escribe el número en el pizarrón: `3 1 1 2 4 5 1 0 9 8 8` y dibuja dos "tijeras" moviéndose por los espacios entre dígitos.
2. "Para partir en 3, necesito 2 puntos de corte. Si llamo i al primero y j al segundo, con $1 \leq i < j \leq 10$, las tres partes son los dígitos antes de i, entre i y j, y después de j."
3. Muestra un ejemplo concreto: $i = 4$, $j = 7$: `3112 + 451 + 0988` $= 3112 + 451 + 988 = 4551$.
4. "¿Cuántas combinaciones (i, j) hay?" Son $\binom{10}{2} = 45$. "¿Podemos probarlas todas? Sí, 45 es un número pequeñísimo."
5. Muestra otra partición: $i = 3$, $j = 7$: `311 + 2451 + 0988` $= 311 + 2451 + 988 = 3750$. "Esta es menor."
6. Pregunta de cierre: "¿Cómo sabes que 3750 es el mínimo de todas las 45?" — dejar para el equipo.

El ejemplo en el enunciado del problema muestra "`3112 + 451 + 0988 = 3750`", pero $3112 + 451 + 988 = 4551 \neq 3750$. La partición correcta que da 3750 es $i = 3$, $j = 7$: "`311 + 2451 + 0988`" ($311 + 2451 + 988 = 3750$). El enunciado tiene un error tipográfico en la descripción del ejemplo. La entrada y la salida correctas (`31124510988` $\rightarrow$ `3750`) sí son correctas. No menciones esto hasta que los alumnos hayan resuelto el problema.

• Análisis de la solución

Leer el número como `string` de longitud 11. Iterar sobre todos los pares (i, j) con $1 \leq i < j \leq 10$:

- Parte 1: `s.substr(0, i)` $\rightarrow a$
- Parte 2: `s.substr(i, j - i)` $\rightarrow b$
- Parte 3: `s.substr(j)` $\rightarrow c$
- Actualizar mínimo de $a + b + c$

Los ceros iniciales (`"0988"`) se eliminan automáticamente al convertir con `stoll`.

El peor caso es “9” + “9” + “999999999” $= 9 + 9 + 999\,999\,999 = 1\,000\,000\,017$, que cabe en `int` (máx $\approx 2.1 \times 10^9$). Usar `long long` es buena práctica defensiva.

• Solución en C++

```
#include <iostream>
#include <string>
#include <climits>
using namespace std;

int main() {
    string s;
    cin >> s;

    long long minSuma = LLONG_MAX;
    int n = (int)s.size();   // debe ser 11

    for (int i = 1; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            long long a = stoll(s.substr(0, i));
            long long b = stoll(s.substr(i, j - i));
            long long c = stoll(s.substr(j));
            minSuma = min(minSuma, a + b + c);
        }
    }

    cout << minSuma << "\n";
}
```

¿Qué pasa si la cadena empieza con cero?
El enunciado dice que es un número de 11 dígitos, así que no empieza con cero. Las subcadenas intermedias sí pueden empezar con cero; `stoll` lo maneja correctamente.

Un alumno lee el número como `long long` en vez de `string`. ¿Puede extraer las partes con aritmética?
Sí, pero es mucho más complicado (divisiones y módulos con potencias de 10). La solución con `string` y `substr` es más clara y equivalente en eficiencia para este tamaño.

¿Por qué el mínimo no siempre es la partición más equilibrada?
Intuitivamente sí: 4+4+3 dígitos distribuye bien y evita números grandes, por eso el mínimo tiende a estar cerca de esa distribución. Pero el valor de los dígitos importa; por eso hay que verificar todas las opciones.

Materiales y recursos

- **Cuaderno:** `sesiones/sesion18_taller_2.ipynb`
- **Notas alumno:** `notas_alumno/pdf/sesion18_taller_2.pdf`